

PRAKTIKUM ARSITEKTUR PERANGKAT LUNAK

Disusun Untuk Memenuhi Tugas
Praktikum Arsitektur Perangkat Lunak

Oleh :

M. IHSAN RIZQULLAH ADEA

(2208107010029)



PROGRAM STUDI INFORMATIKA
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
UNIVERSITAS SYIAH KUALA
DARUSSALAM, BANDA ACEH

2025

Studi Kasus: Sistem Informasi Menu dan Pemesanan Makanan di Kantin Universitas

Deskripsi:

Sebuah kantin di lingkungan universitas ingin menerapkan sistem digital untuk mempermudah mahasiswa dan staf. Melalui aplikasi atau situs web, sistem harus **menampilkan daftar menu makanan dan minuman yang tersedia beserta harganya (R01)** secara *real-time*. Pengguna kemudian dapat **memilih dan menambahkan item menu yang mereka inginkan ke dalam keranjang pesanan (R02)**. Setelah selesai memilih, pengguna akan memasukkan nama mereka lalu sistem akan **menyimpan pesanan tersebut dan secara otomatis menghasilkan nomor antrean unik (R03)**. Sementara itu, untuk memastikan data menu selalu akurat, **staf kantin diberikan akses untuk dapat memperbarui status ketersediaan menu, seperti mengubahnya menjadi 'tersedia' atau 'habis' (R04)**. Staf juga dapat **melihat daftar semua pesanan yang masuk secara berurutan (R05)** untuk mempercepat proses penyajian. Pembayaran akan tetap dilakukan secara manual di kasir dengan menunjukkan nomor antrean.

1. Requirements (R):

- (R01) Sistem harus menampilkan daftar menu makanan dan minuman yang tersedia beserta harganya.
- (R02) Pengguna dapat memilih dan menambahkan item menu ke dalam keranjang pesanan.
- (R03) Setelah selesai memilih, sistem akan menyimpan pesanan dan menghasilkan nomor antrean untuk pengguna.
- (R04) Staf kantin dapat memperbarui status ketersediaan menu (tersedia/habis).
- (R05) Staf kantin dapat melihat daftar pesanan yang masuk secara berurutan.

2. Facts (F):

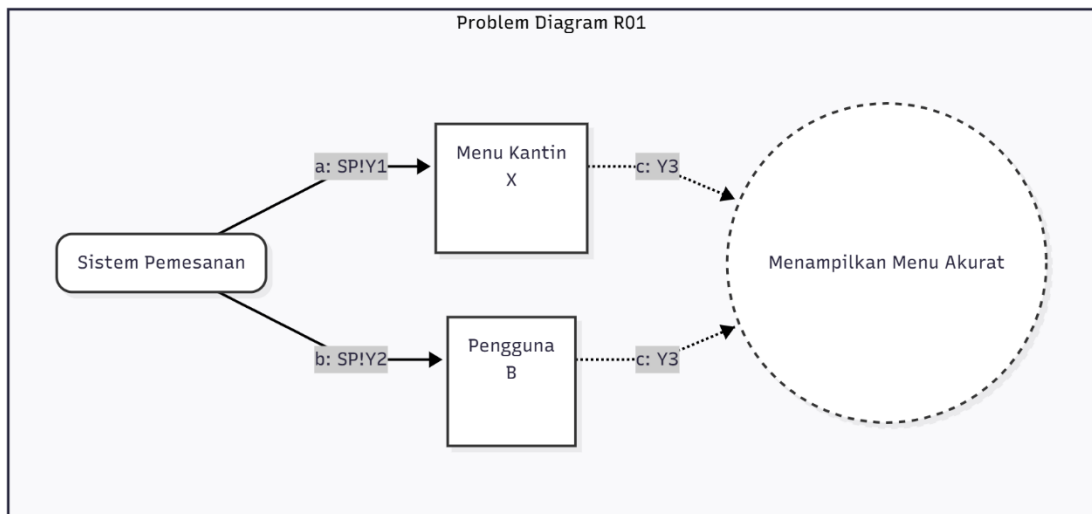
- (F01) Setiap item menu memiliki satu harga yang pasti.
- (F02) Setiap pengguna hanya dapat memiliki satu keranjang belanja pada satu waktu.
- (F03) Makanan yang sudah habis tidak dapat dipesan lagi.
- (F04) Setiap pesanan yang berhasil dibuat bersifat unik dan memiliki nomor antrean yang berbeda.
- (F05) Waktu operasional kantin terbatas pada jam tertentu setiap harinya.

3. Assumptions (A):

- (A01) Pengguna memiliki koneksi internet untuk mengakses sistem.
- (A02) Pengguna akan membayar pesannya di kasir saat pengambilan makanan.
- (A03) Staf kantin akan secara rutin dan akurat memperbarui status ketersediaan menu.
- (A04) Pengguna akan segera mengambil pesannya setelah pesanan tersebut siap.
- (A05) Tidak ada pembatalan pesanan setelah pesanan berhasil dikonfirmasi oleh sistem.

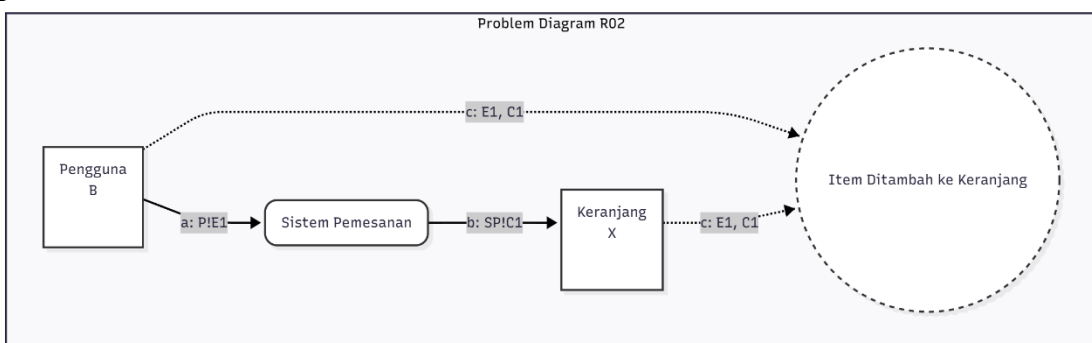
Problem Frames: Sistem Informasi Menu dan Pemesanan Makanan di Kantin Universitas

1. (R01) Sistem harus menampilkan daftar menu makanan dan minuman yang tersedia beserta harganya.



Keterangan

- Machine: Sistem Pemesanan (SP)
 - Domains: Menu Kantin (X - Lexical) dan Pengguna (B - Biddable).
 - Fenomena:
 - a: SP!Y1: Sistem membaca data dari domain Menu Kantin.
 - b: SP!Y2: Sistem menampilkan data ke Pengguna.
 - Requirement (c: Y3): Memastikan menu yang dilihat Pengguna sesuai dengan yang ada di Menu Kantin.
2. (R02) Pengguna dapat memilih dan menambahkan item menu ke dalam keranjang pesanan.

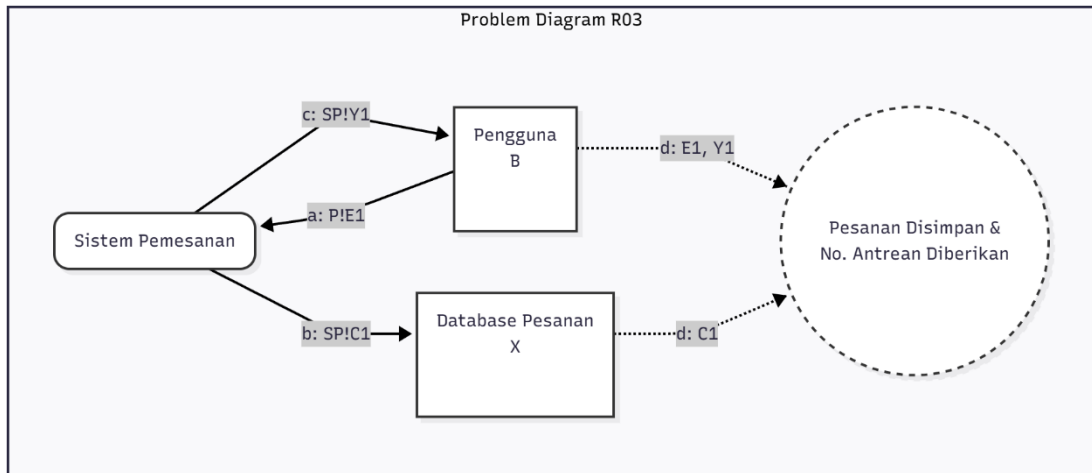


Keterangan

- Machine: Sistem Pemesanan (SP)
- Domains: Pengguna (P) dan Keranjang (X - Lexical, merepresentasikan data keranjang).
- Fenomena:
 - a: P!E1: Pengguna melakukan aksi (event) menambah item.

- b: SP!C1: Sistem melakukan kontrol untuk mengubah data di Keranjang.
- Requirement (c: E1, C1): Memastikan aksi dari Pengguna menyebabkan perubahan yang benar pada Keranjang.

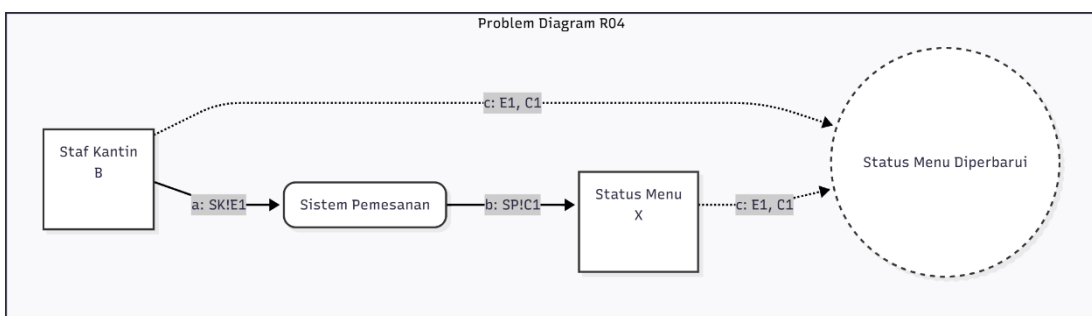
3. (R03) Setelah selesai memilih, sistem akan menyimpan pesanan dan menghasilkan nomor antrean untuk pengguna.



Keterangan

- Machine: Sistem Pemesanan (SP)
- Domains: Pengguna (P) dan Database Pesanan (X).
- Fenomena:
 - a: P!E1: Pengguna mengonfirmasi pesanan.
 - b: SP!C1: Sistem menyimpan data ke Database Pesanan.
 - c: SP!Y1: Sistem menampilkan nomor antrean ke Pengguna.
- Requirement (d): Memastikan konfirmasi dari Pengguna menghasilkan data yang tersimpan dan nomor antrean yang ditampilkan.

4. (R04) Staf kantin dapat memperbarui status ketersediaan menu (tersedia/habis).

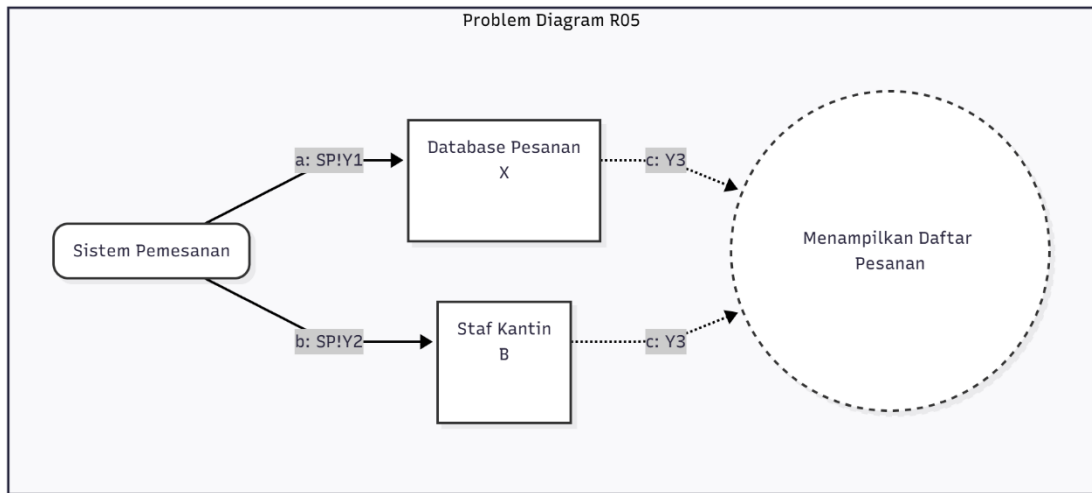


Keterangan

- Machine: Sistem Pemesanan (SP)
- Domains: Staf Kantin (SK) dan Status Menu (X).
- Fenomena:
 - a: SK!E1: Staf Kantin memberi perintah update.
 - b: SP!C1: Sistem mengubah data pada Status Menu.

- Requirement (c): Memastikan perintah dari Staf menyebabkan perubahan yang benar pada data menu.

5. (R05) Staf kantin dapat melihat daftar pesanan yang masuk secara berurutan.

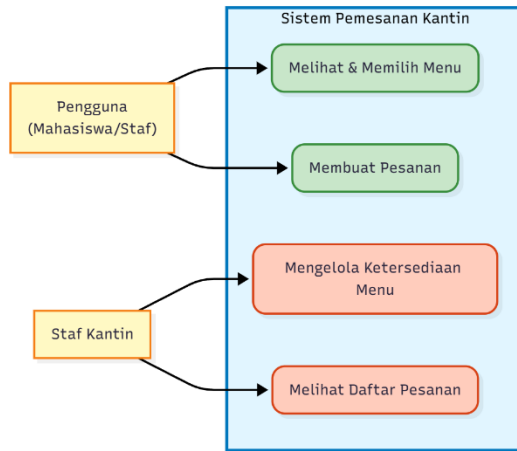


Keterangan

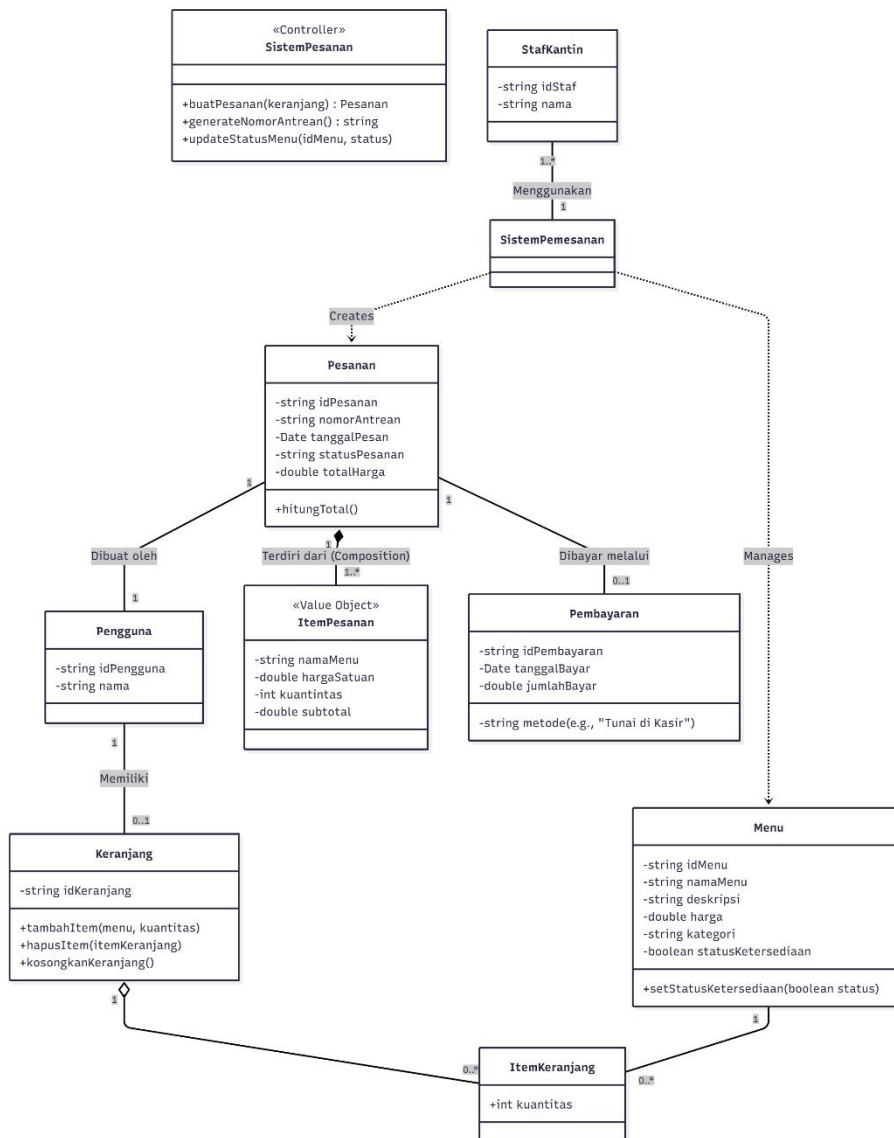
- Machine: Sistem Pemesanan (SP)
- Domains: Database Pesanan (X) dan Staf Kantin (B).
- Fenomena:
 - a: SP!Y1: Sistem membaca data dari Database Pesanan.
 - b: SP!Y2: Sistem menampilkan data ke Staf Kantin.
- Requirement (c: Y3): Memastikan data pesanan yang ditampilkan ke Staf sesuai dengan yang ada di database.

UML: Sistem Informasi Menu dan Pemesanan Makanan di Kantin Universitas

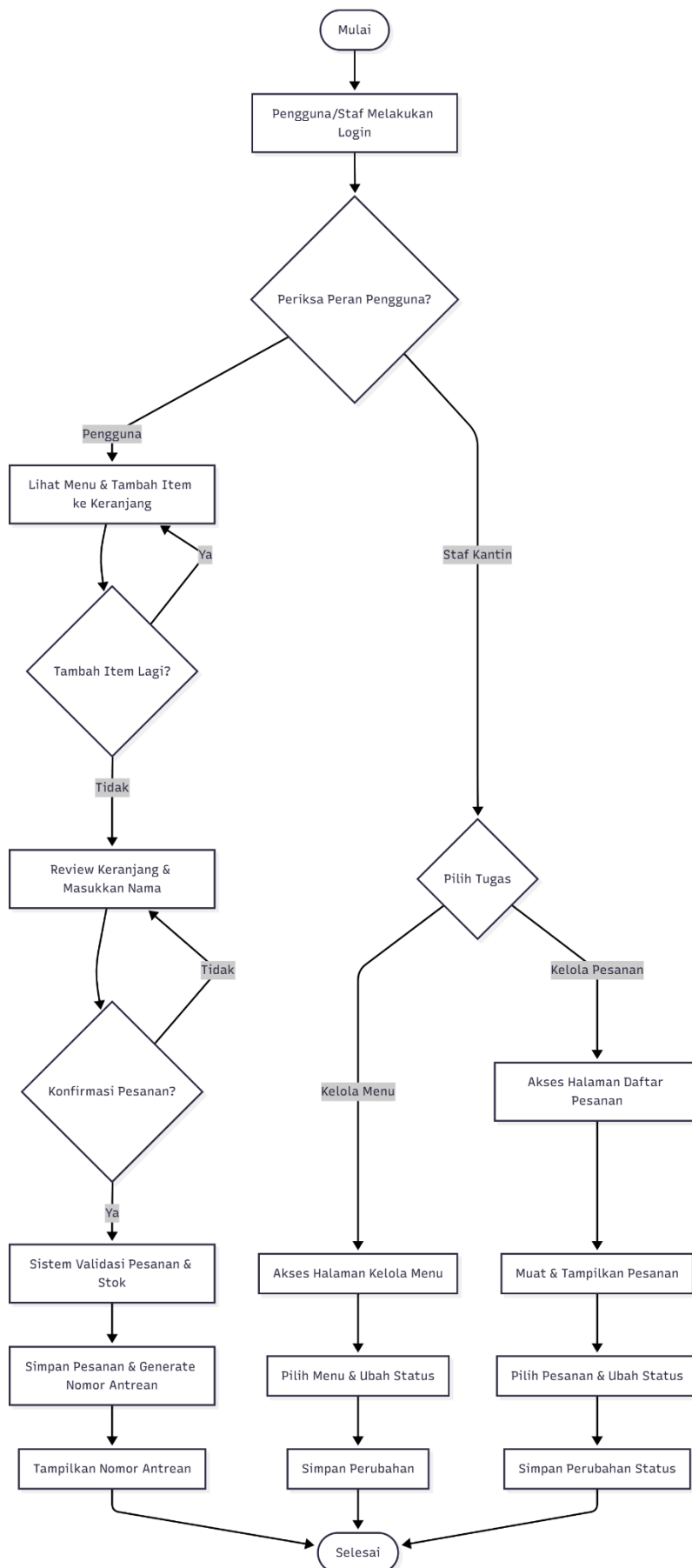
1. Use Case Diagram



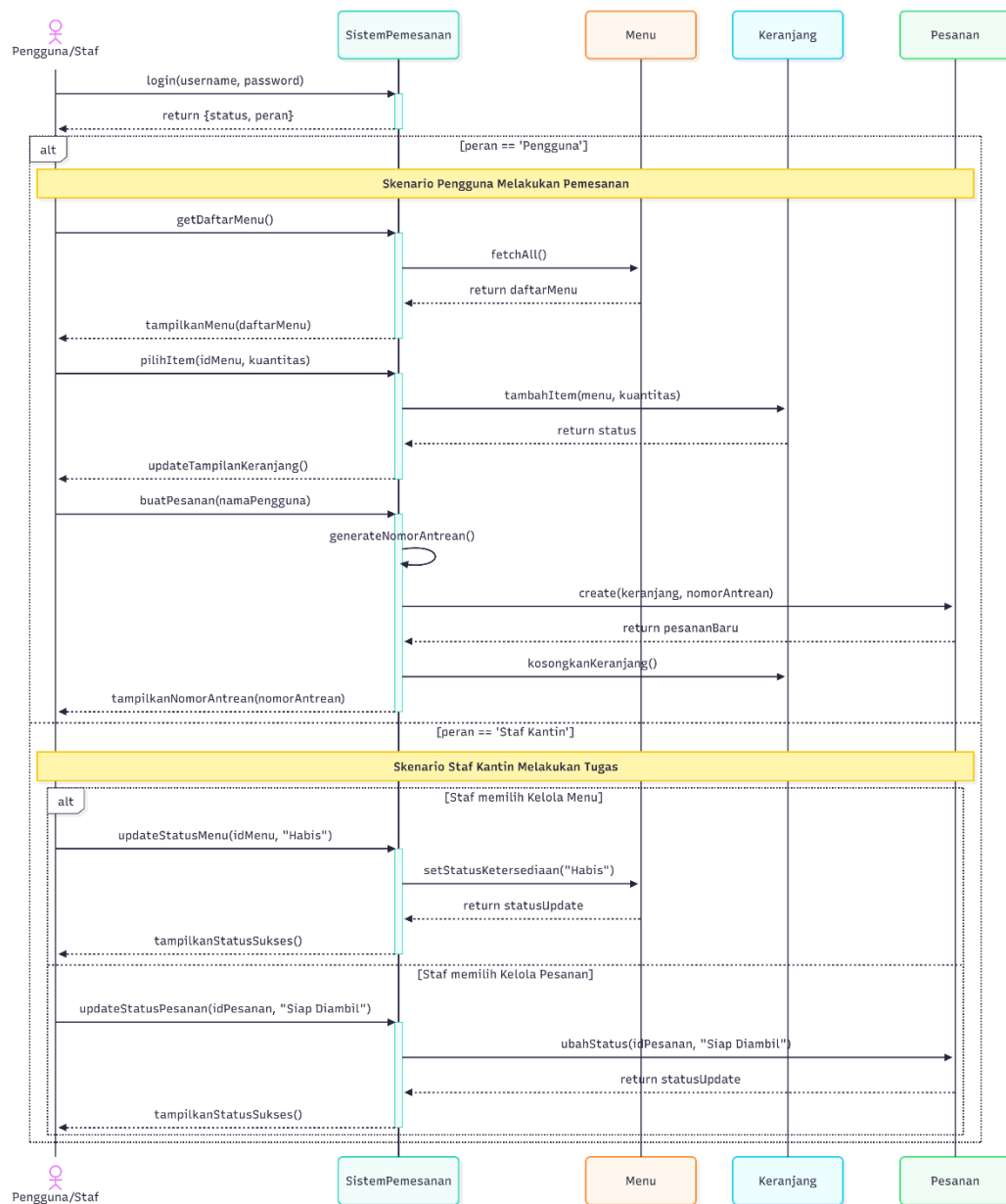
2. Class Diagram



3. Activity Diagram



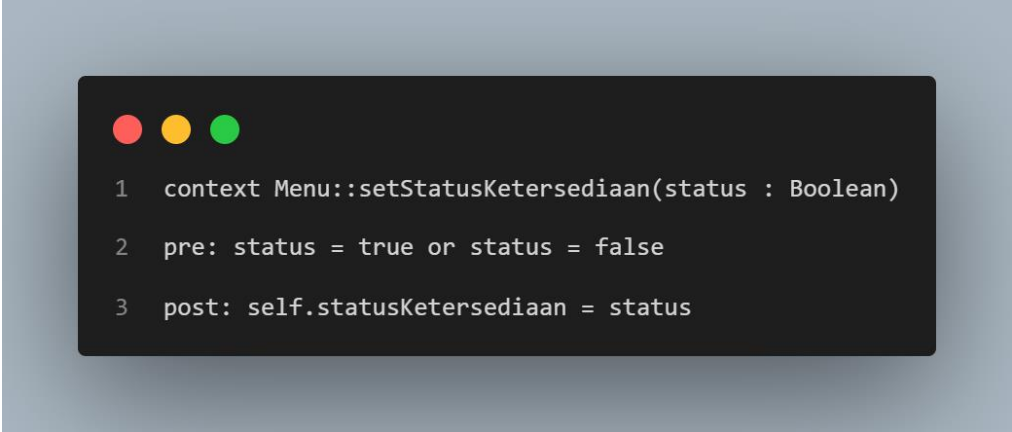
4. Sequence Diagram



OCL: Sistem Informasi Menu dan Pemesanan Makanan di Kantin Universitas

1. Class Menu

- Konteks: Mendefinisikan syarat dan hasil untuk method setStatusKetersediaan pada class Menu.

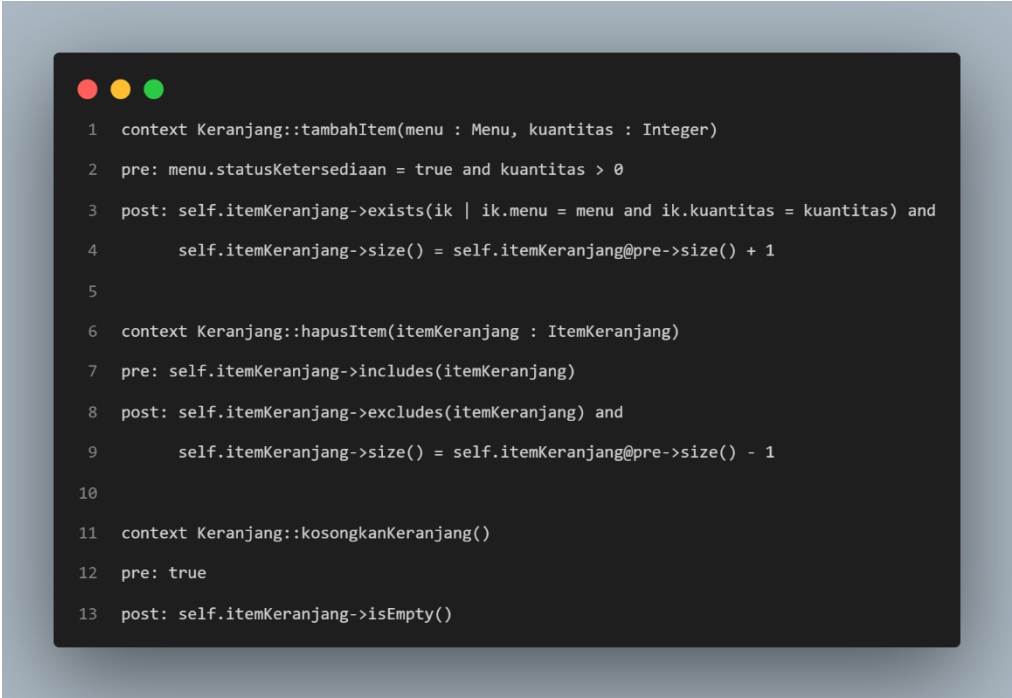


```
1 context Menu::setStatusKetersediaan(status : Boolean)
2 pre: status = true or status = false
3 post: self.statusKetersediaan = status
```

- Penjelasan:
 - pre: Sebelum method dijalankan, memastikan nilai status yang dimasukkan adalah nilai boolean yang valid (true atau false).
 - post: Setelah method berhasil dijalankan, memastikan atribut statusKetersediaan dari objek Menu tersebut telah berubah sesuai dengan nilai status yang baru.

2. Class Keranjang

- Konteks: Mendefinisikan syarat dan hasil untuk method-method yang ada di dalam class Keranjang.



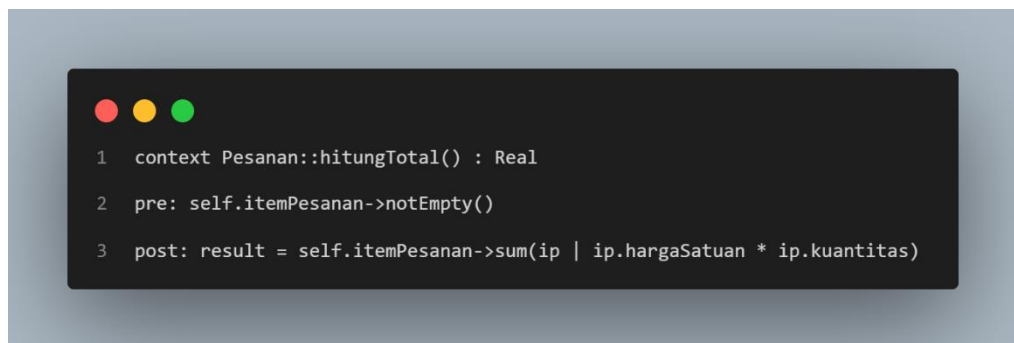
```
1 context Keranjang::tambahItem(menu : Menu, kuantitas : Integer)
2 pre: menu.statusKetersediaan = true and kuantitas > 0
3 post: self.itemKeranjang->exists(ik | ik.menu = menu and ik.kuantitas = kuantitas) and
4       self.itemKeranjang->size() = self.itemKeranjang@pre->size() + 1
5
6 context Keranjang::hapusItem(itemKeranjang : ItemKeranjang)
7 pre: self.itemKeranjang->includes(itemKeranjang)
8 post: self.itemKeranjang->excludes(itemKeranjang) and
9       self.itemKeranjang->size() = self.itemKeranjang@pre->size() - 1
10
11 context Keranjang::kosongkanKeranjang()
12 pre: true
13 post: self.itemKeranjang->isEmpty()
```

- Penjelasan:

- `tambahItem`: Memastikan menu yang ditambahkan harus tersedia dan kuantitasnya lebih dari 0 (pre). Setelah itu, memastikan item tersebut benar-benar ada di dalam keranjang dan jumlah item di keranjang bertambah (post).
- `hapusItem`: Memastikan item yang akan dihapus memang ada di dalam keranjang (pre). Setelah itu, memastikan item tersebut sudah tidak ada lagi dan jumlah item berkurang (post).
- `kosongkanKeranjang`: Tidak ada syarat khusus (pre: true). Setelah dijalankan, memastikan keranjang benar-benar kosong (post).

3. Class Pesanan

- Konteks: Mendefinisikan syarat dan hasil untuk method `hitungTotal` pada class `Pesanan`.



```

1 context Pesanan::hitungTotal() : Real
2 pre: self.itemPesanan->notEmpty()
3 post: result = self.itemPesanan->sum(ip | ip.hargaSatuan * ip.kuantitas)

```

- Penjelasan:
 - pre: Sebelum menghitung total, memastikan ada item di dalam pesanan.
 - post: Memastikan nilai yang dikembalikan (result) adalah hasil dari penjumlahan `hargaSatuan` dikali `kuantitas` untuk setiap item di dalam pesanan.

4. Class SistemPemesanan (Controller)

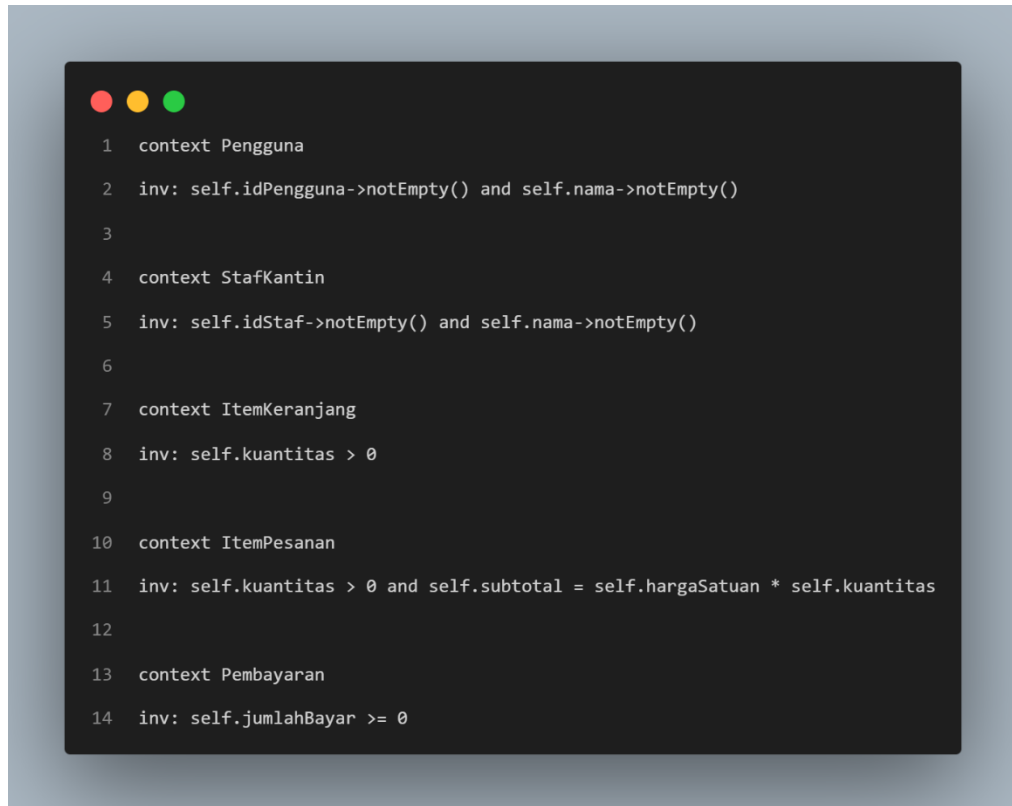
- Konteks: Mendefinisikan syarat dan hasil untuk method-method utama pada class SistemPemesanan.

```
1 context SistemPemesanan::buatPesanan(keranjang : Keranjang) : Pesanan
2 pre: keranjang.itemKeranjang->notEmpty()
3 post: result.oclIsNew() and
4       Pesanan.allInstances()->includes(result) and
5       keranjang.itemKeranjang->isEmpty()
6
7 context SistemPemesanan::generateNomorAntrean() : String
8 pre: true
9 post: Pesanan.allInstances()@pre->forAll(p | p.nomorAntrean <> result)
10
11 context SistemPemesanan::updateStatusMenu(idMenu : String, status : Boolean)
12 pre: Menu.allInstances()->exists(m | m.idMenu = idMenu)
13 post: let menuDiubah = Menu.allInstances()->any(m | m.idMenu = idMenu) in
14       menuDiubah.statusKetersediaan = status
```

- Penjelasan:
 - buatPesanan: Memastikan keranjang tidak kosong (pre). Setelah itu, memastikan sebuah objek Pesanan baru telah dibuat, pesanan tersebut masuk ke dalam daftar semua pesanan, dan keranjang yang tadi digunakan menjadi kosong (post).
 - generateNomorAntrean: Tidak ada syarat khusus (pre). Memastikan nomor antrean yang dihasilkan (result) belum pernah digunakan oleh pesanan lain sebelumnya (post).
 - updateStatusMenu: Memastikan menu dengan idMenu yang dituju memang ada (pre). Setelah itu, memastikan status ketersediaan dari menu tersebut benar-benar telah diperbarui (post).

5. Class Tanpa Method (Invariants)

- Konteks: Untuk memenuhi syarat "satu OCL per class", class yang tidak memiliki method akan diberikan Invariant (aturan yang harus selalu benar) sederhana berdasarkan atributnya.



```
1 context Pengguna
2 inv: self.idPengguna->notEmpty() and self.nama->notEmpty()
3
4 context StafKantin
5 inv: self.idStaf->notEmpty() and self.nama->notEmpty()
6
7 context ItemKeranjang
8 inv: self.kuantitas > 0
9
10 context ItemPesanan
11 inv: self.kuantitas > 0 and self.subtotal = self.hargaSatuan * self.kuantitas
12
13 context Pembayaran
14 inv: self.jumlahBayar >= 0
```

- Penjelasan:
 - Pengguna & StafKantin: Memastikan ID dan nama tidak boleh kosong.
 - ItemKeranjang & ItemPesanan: Memastikan kuantitas item selalu lebih dari nol. Untuk ItemPesanan, juga memastikan subtotal dihitung dengan benar.
 - Pembayaran: Memastikan jumlah pembayaran tidak boleh bernilai negatif.

Decorator: Sistem Informasi Menu dan Pemesanan Makanan di Kantin Universitas

1. Masalah Awal (Desain Sebelum Decorator)

Kita akan berfokus pada class Pesanan.

Bayangkan kita ingin menambah fungsionalitas di mana pengguna bisa meminta opsi tambahan saat membuat pesanan. Opsi-opsi ini memengaruhi total harga dan deskripsi pesanan.

Contoh Opsi Tambahan:

- "Sambal Tambahan" (+ Rp1.500)
- "Kemasan Khusus" (dibungkus terpisah, + Rp2.000)
- "Pesanan Prioritas" (dikerjakan lebih dulu, + Rp3.000)

Jika kita menggunakan pendekatan pewarisan (inheritance) untuk menyelesaikan ini, kita akan membuat subclass baru dari Pesanan untuk setiap variasi.

- PesananDenganSambal (mewarisi dari Pesanan)
- PesananKemasanKhusus (mewarisi dari Pesanan)
- PesananPrioritas (mewarisi dari Pesanan)

Masalah utamanya muncul ketika pengguna ingin menggabungkan opsi-opsi ini.

- Bagaimana jika pengguna ingin "Sambal Tambahan" DAN "Kemasan Khusus"?
- Bagaimana jika mereka ingin ketiga opsi sekaligus?

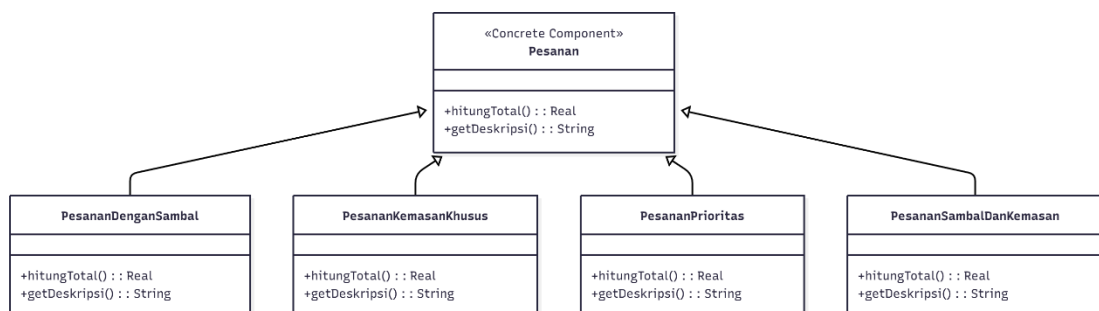
Dengan inheritance, kita terpaksa membuat class baru untuk setiap kombinasi yang mungkin:

- PesananSambalDanKemasan
- PesananSambalDanPrioritas
- PesananKemasanDanPrioritas
- PesananSambalKemasanPrioritas

Hal ini menyebabkan "ledakan jumlah kelas" (combinatorial explosion). Desain ini menjadi sangat kaku, sulit dirawat, dan tidak fleksibel jika kita ingin menambahkan opsi baru (misalnya, "Tanpa MSG") di kemudian hari.

UML Class Diagram Awal (Sebelum Decorator)

Diagram berikut mengilustrasikan masalah "ledakan jumlah kelas" yang disebabkan oleh penggunaan inheritance murni.



Penjelasan

Diagram ini menunjukkan Pesanan sebagai parent class. Setiap opsi tambahan, dan setiap kombinasi dari opsi tersebut, harus dibuat sebagai subclass baru yang meng-

override method `hitungTotal()` dan `getDeskripsi()`. Ini jelas tidak efisien dan sulit dikelola.

2. Solusi dengan Decorator Pattern

Decorator Pattern menyelesaikan masalah "ledakan jumlah kelas" dengan mengubah pendekatan secara fundamental: dari pewarisan (inheritance) menjadi komposisi (composition).

Alih-alih membuat subclass baru untuk setiap kombinasi fitur, kita akan "membungkus" (wrap) objek Pesanan dasar dengan lapisan-lapisan decorator. Setiap decorator bertugas menambahkan satu fungsionalitas spesifik (satu opsi tambahan) secara dinamis.

Untuk menerapkannya pada Sistem Pemesanan Kantin, kita akan mendefinisikan elemen-elemen penting berikut:

- **Komponen Dasar (Interface Utama):** Kita akan membuat sebuah interface baru bernama `IPesanan`. Interface ini akan mendefinisikan method utama yang akan dimiliki oleh pesanan dasar maupun semua lapisannya, yaitu `hitungTotal(): Real` dan `getDeskripsi(): String`.
- **Komponen Konkret (Concrete Component):** Ini adalah class. Class ini akan kita modifikasi agar mengimplementasikan interface `IPesanan`. Ini adalah objek dasar yang akan "dihias" atau "dibungkus".
- **Base Decorator (Class Pembungkus Dasar):** Kita akan membuat sebuah abstract class bernama `PesananDecorator`. Class ini juga mengimplementasikan `IPesanan`. Yang terpenting, class ini memiliki referensi (komposisi) ke objek `IPesanan` lain yang dibungkusnya (wrappee). Method-nya (`hitungTotal()` dan `getDeskripsi()`) akan mendelegasikan pemanggilan ke objek yang dibungkusnya.
- **Concrete Decorators (Class-Class Tambahan):** Ini adalah class-class spesifik untuk setiap opsi tambahan yang kita identifikasi, misalnya: `SambalDecorator`, `KemasanKhususDecorator`, dan `PrioritasDecorator`. Setiap class ini akan mewarisi dari `PesananDecorator` dan meng-override method `hitungTotal()` (untuk menambahkan biayanya sendiri ke total) dan `getDeskripsi()` (untuk menambahkan deskripsi opsinya).

Fleksibilitas Desain

Pendekatan ini membuat sistem menjadi jauh lebih fleksibel. Kita dapat mengombinasikan opsi dalam urutan apa pun saat runtime tanpa perlu membuat class baru untuk setiap kombinasi.

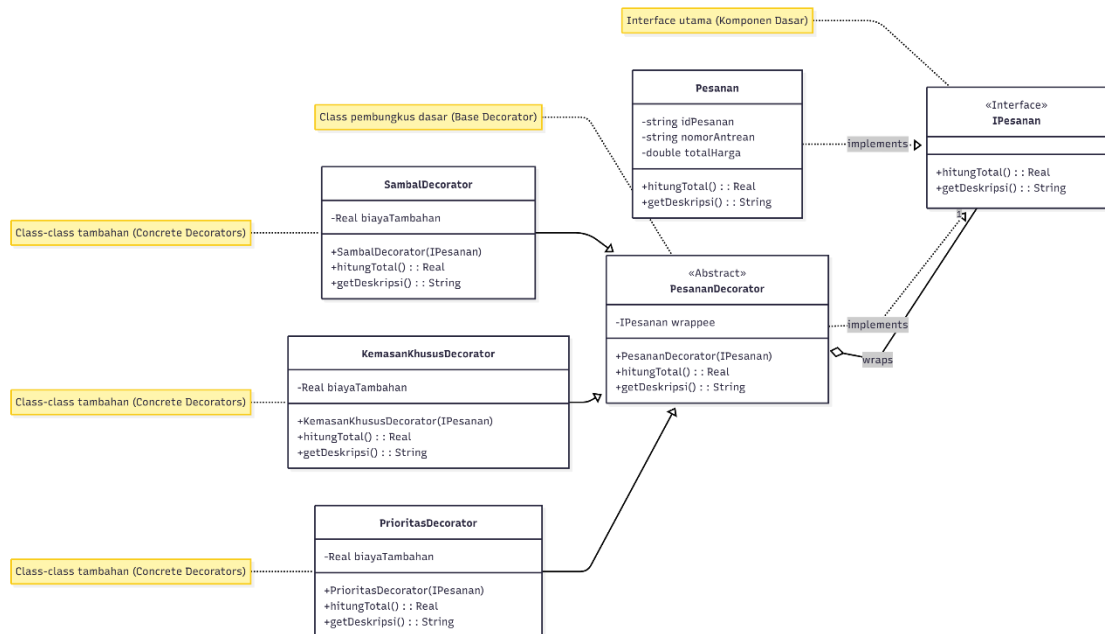
Misalnya, jika seorang pengguna ingin memesan dengan tambahan sambal dan kemasan khusus, kode kita tidak perlu mencari class `PesananSambalDanKemasan`. Sebaliknya, kita cukup "membungkus" objeknya secara dinamis:

```
pesananBaru = new SambalDecorator( new KemasanKhususDecorator( new Pesanan()  
));
```

Jika di masa depan kita ingin menambahkan opsi baru (misalnya, TanpaMsgDecorator), kita cukup membuat satu class decorator baru tanpa perlu mengubah kode di class Pesanan atau decorator lainnya yang sudah ada. Ini menyelesaikan masalah "ledakan jumlah kelas" dan membuat kode lebih mudah dirawat.

3. Desain UML Class Diagram (Setelah Menggunakan Decorator)

Berikut adalah Class Diagram yang telah diperbarui untuk mengimplementasikan Decorator Pattern pada class Pesanan. Diagram ini menambahkan interface IPesanan dan beberapa class decorator baru, yang memungkinkan penambahan opsi secara dinamis.



Penjelasan

- **IPesanan** adalah interface utama yang mendefinisikan operasi dasar.
- **Pesanan** adalah class konkret yang mengimplementasikan **IPesanan**. Ini adalah objek dasar yang akan kita bungkus.
- **PesananDecorator** adalah class abstract pembungkus. Ia juga mengimplementasikan **IPesanan** dan "memegang" (wraps) objek **IPesanan** lain.
- **SambalDecorator**, **KemasanKhususDecorator**, dan **PrioritasDecorator** adalah class tambahan yang sesungguhnya. Mereka "membungkus" **Pesanan** (atau decorator lain) untuk menambahkan biaya dan deskripsi baru secara dinamis.

4. Penjelasan Singkat Alur

Penerapan Decorator Pattern ini mengubah cara sistem menangani opsi tambahan pada pesanan. Alih-alih mencari class yang berbeda untuk setiap kombinasi pesanan, sistem akan menggunakan proses pembungkusan (wrapping) yang dinamis.

Saat seorang pengguna menyelesaikan pesanan dasarnya dan mulai menambahkan opsi seperti "Sambal Tambahan", sistem tidak akan mengubah objek **Pesanan** asli. Sebaliknya, sistem akan membuat sebuah objek **SambalDecorator** baru dan

"membungkus" objek Pesanan tersebut di dalamnya. Jika pengguna kemudian menambahkan "Kemasan Khusus", sistem akan sekali lagi membungkus decorator sebelumnya dengan KemasanKhususDecorator, menciptakan tumpukan lapisan.

Ketika sistem perlu menghitung total harga, ia akan memanggil method `hitungTotal()` pada lapisan terluar (`KemasanKhususDecorator`). Decorator ini akan memanggil `hitungTotal()` pada objek yang dibungkusnya (`SambalDecorator`), yang kemudian akan memanggil `hitungTotal()` pada Pesanan dasar. Setelah Pesanan dasar mengembalikan harga awalnya, setiap lapisan decorator akan menambahkan biayanya sendiri secara berurutan saat pemanggilan kembali ke atas. Proses yang sama berlaku untuk `getDeskripsi()`, di mana setiap lapisan menambahkan deskripsi opsinya sendiri ke deskripsi dasar.